ktunotes.in

GET IT ON Google Play



KTU NOTES

The learning companion.

KTU STUDY MATERIALS | SYLLABUS | LIVE NOTIFICATIONS | SOLVED QUESTION PAPERS

Website: www.ktunotes.in

TUPLES

- A Tuple is an ordered set of values, where each value is identified by an index.
- The values that make up a tuple are called its elements.
- Individual elements can be of any type, even another tuple.
- Tuple is Immutable .Because we can't modify the elements in a Tuple

CREATING A TUPLE

- A Tuple created by enclosing the all the elements inside parenthesis () separated by commas

  tuple name=(element1,element2,…….)

  Example: a= (10, 20, 30, 40)

  s= ("Python","Java","C")

  t= ("hello" , 2.0, 5, [10, 20] , ("Python","Java"))

  - The last tuple contains a string, a float, an integer, list and another tuple
  - If a tuple contains another tuple it is known as nested tuple. Here t is a nested tuple.

- A Tuple that contains no elements is called the empty tuple.

  Eg: b = ()

DISPLAYING A TUPLE

Eg: numbers=(1,2,3,4,5)

print numbers Output: (1,2,3,4,5)

DOWNLOADED FROM KTUNOTES.IN

ACCESSING TUPLE ELEMENTS

- The elemets are accessed by using index numbers. The index start at 0.
- The index of first element is 0,second element is 1,third element is 2 and so on.
- If tuple contains ‘n’ elements index number varies from 0 to (n-1)
- Python allows negative indexes for tuple elements. The negative index starts from -1(last element)
- The negative index of last element is -1,second last element is -2, and so on.

Eg1: **a=(2,4,8,12)**

-ve index	-4	-3	-2	-1
+ve Index	0	1	2	3
Element	2	4	8	12

Print a[2] **Output: 8**

print a[-3] **Output :4**

DOWNLOADED FROM KTUNOTES.IN

Eg2:T1=("Happy" , 20 , [2,10,1,5] , (44,55))

print T1 [2] **Output: (2,10,1,5)**

Print Tl[1] **Output :20**

Print T1[2] [1] **Output:10**

Print T1[3] **Output (44,55)**

Print T1[0][1] **Output :a**

TUPLE LENGTH

- **The function len is used to find the length of a TUPLE(number of elements in the TUPLE)**

Syntax: len(tuple name)

Eg1: t1=(20,30,40,50,60)

l=len(t1) **l=5**

Eg2: t1=("Happy" , 20 , [2,10,1,5])

l=len(t1) **l=3**

TUPLE TRAVERSAL

- **A lot of computations involve processing a list one element at a time.**
- **We may start at the beginning, select each element in turn, and continue until the end. This pattern of processing is called a traversal.**
- **Traversal can be done with while or for loops.**

DOWNLOADED FROM KTUNOTES.IN

Eg: a=(11,22,33,44)

Using while loop

```
i=0
    while i<len(a)
        print a[i]
        i=i+1
```

Using for loop

```
i)for i in range (0,len(a)):
    print  a[i]
```

Easy method

```
ii)for i in a:
    print i
```

Output: 11

22

33

44

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

TUPLE SLICING

- A slice is a subset of a tuple .
- Slice operator (:) is used to extract a slice of the tuple

Syntax: tuple name[n : m : stepsize]

returns the subset of the list from the nth element(including) to the mth element(excluding)

Eg: s=(1,2,3,4,5,6,7,8)

1.s[0:5:1] Output: (1,2,3,4,5)

2.s[3:7:1] Output: (4,5,6,7)

3.s[2:6:2] Output: (3, 5)

4.s[1:4] Output: (2,3,4)

5.s[-2:-7:-1] Output: (7,6,5,4,3)

6.s[-5:-2] Output : (4,5,6)

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

- If you omit the first index (before the colon), the slice starts from beginning of the tuple. If you omit the second index, the slice goes to the end of the tuple.(+ve step size)
- If you omit the first index (before the colon), the slice starts from end of the tuple. If you omit the second index, the slice goes to the beginning of the tuple.(-ve step size)

s=(“Hello” , 11 , 5 , 3 , 4.5 , ’x’ ,10)

1.s[:5:1] : (“hello”, 11, 5, 3, 4.5)

2.s[:4:] : (“hello”, 11, 5, 3)

3.s[2::2] : (5,4.5,10)

4.s[2::] : (5 , 3 , 4.5 , ’x’ ,10)

5.s[2::-1] : (5,11,,”Hello”)

5.s[::] : (“Hello” , 11 , 5 , 3 , 4.5 , ’x’ ,10)

6.s[::-1] : (10, ’x’ , 4.5 , 3 , 5 , 11 , ”Hello”)

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

CHANGING A TUPLE

- **Unlike lists, tuples are immutable.**
- **This means that elements of a tuple cannot be changed . But, if the element is itself a mutable data type like list, its nested items can be changed.**

Eg: x=(1,2,5, [3,8,9])

x[3][1] =88 Now,x : (1,2,5, [3,88,9])

DELETING TUPLE ELEMENTS

- **we can delete an entire tuple**

Syntax : del tuple name

Eg: x=(2,5,3,8,9)

del x will delete the entire tuple.

- **we cannot change or delete the elements in a tuple. But if the element is itself a mutable data type like list, its nested items can be deleted.**

Eg: x=(1,2,5, [3,8,9])

del x[3][1] Now,x : (1,2,5, [3,9])

DOWNLOADED FROM KTUNOTES.IN

TUPLE OPERATIONS

- **There are 4 basic tuple operations in Python**

1. concatenation 2.Repetition 3. in 4. not in

1. concatenation (+)

- **Joins two tuples**

Eg: x=(1,2,3,4)

y=(10,20,30)

z=x + y z: (1,2,3,4,10,20,30)

Ktunotes.in

2.Repetition (*)

- **This operator replicates the elements in the tuple**

Eg: a=(1,2,3)

b=3*a b : (1,2,3,1,2,3,1,2,3)

DOWNLOADED FROM KTUNOTES.IN

3. in

- Used to check whether an element present in a tuple.
- Returns true if element is present .Otherwise returns false.

Eg1: X=(2,5,3,8,9)

```
if(2 in X):
    print ”Present”
else:
    print”Not Present”
```

4. not in

- Used to check whether an element not present in a tuple
- Returns true if element is not present .Otherwise returns false.

Eg: X=(2,5,3,8,9)

```
if(2 not in X):
    print ”Not Present”
else:
    print”Present”
```


Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

BUILT IN TUPLE METHODS (FUNCTIONS)

count()

- This method is used to count the no.of occurences of an element in a tuple

Syntax: tuplename. count(element)

Eg: a=(1,2,3,4,3,5,1,9,1)

b= a.count(1)

b=3

index()

- This method is used to find the lowest index of an element in a tuple

Syntax: tuplename. index(element)

Eg: a=(1,2,3,4,3,5,1,9,1)

b= a.index(3)

b=2

Ktunotes.in

max()

- This method is used to find the largest element in a tuple.

Syntax: max(tuple name)

Eg: a= (33,28,99,10)

b=max(a) b:99

min()

- This method is used to find the smallest element in a tuple.

Syntax: min(tuple name)

Eg: a= (33,28,99,10)

b=min(a) b:10

DOWNLOADED FROM KTUNOTES.IN

Sorted()

- **This method is used to store the sorted elements of a tuple in another tuple**

  Syntax: tuple1=sorted(tuple2) : By default the elements are stored in asc. order

  Eg: a=(“C”, ”Python” ,”Java”)

  b=sorted(a)

  Then b = (“C”, ”Java” , ”Python”)

- **For sorting in descending order**

  Syntax: tuple1=sorted(tuple2,reverse=True)

  Eg: a=(“C”, ”Python” ,”Java”)

  b=sorted(a, reverse = True)

  Then b = (“Python”, ”Java” , ”C”)

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

Advantages of Tuple over List

Since tuples are quite similar to lists, both of them are used in similar situations. However, there are certain advantages of implementing a tuple over a list. Below listed are some of the main advantages:

- **Since tuples are immutable, operations with a tuple is faster than with list.**
- **Tuples that contain immutable elements can be used as a key for a dictionary. With lists, this is not possible.**
- **If you have data that doesn't change, implementing it as tuple will guarantee that it remains write-protected.**

Python Program to Swap Two Numbers using Tuple

```
a= input("Enter the first number :")
b=input("Enter the second number :")
print “Before swap: a=", a, "b=",b
(a,b)=(b,a)
print“ Afetr Swap: a=",a, "b=",b
```

TUPLE PACKING & UNPACKING

Tuple packing refers to assigning multiple values into a tuple.

Eg:cstudent = (“Surya” , 10, 249) This tuple stores the name,roll no and mark of a student

Tuple unpacking refers to extracting tuple values back into multiple variables.

Eg: (n,r,m) = student

The tuple values are assigned to the variables n,r,m in that order

ie n= Surya”

r=10

m=249

Largest and smallest element in a set of numbers using list

```
x = []
n=input(“How many numbers?”)
print "\nEnter the numbers"
for i in range(1,n+1):
    k=raw_input()
    x.append(k)
print “List is:” ,x
l=max(x)
s=min(x)
Print “Largest =“ , l , “Smallest=“,s
```


DOWNLOADED FROM KTUNOTES.IN

STRINGS IN PYTHON

- A string datatype is used to store a group of characters.Group of chatacters can be encosed within Singe quotes or double quotes.

Eg: s1= “Hello World”

s2= ‘Python’

s3=‘ ’

s4=“ ”

name=“Johnson”

s1,s2,s3,s4,name are string variables

READING A STRING FROM KEYBOARD

- raw_input() is used in Python2

Eg: s= raw_input(“Enter a string”)

The string entered from keyboard is stored in the variable ‘s’

- In Python3, input() is used

Eg: s= input(“enter a string”)

DOWNLOADED FROM KTUNOTES.IN

ACCESSING INDIVIDUAL CHARACTERS IN THE STRING

- **The index values of characters starts from 0. .**

Eg: s1= “Hello World”

-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d

- VE INDEX NOS

+ VE INDEX NOS

s1[0] =‘H’ s1[1]=‘e’ etc

Ktunotes.in

- **we can use negative indices, which count backward from the end of the string.**

s1[-1] =‘d’ , s1[-2] =‘l’ and so on.

STRING LENGTH

- **The len is a built-in function that returns the number of characters in a string(string length) .**

Eg: s= “Program”

n=len(s)

The value of n will be 7

DOWNLOADED FROM KTUNOTES.IN

TRAVERSAL IN THE STRING

- A lot of computations involve processing a string one character at a time.
- We may start at the beginning, select each character in turn, do something to it, and continue until the end. This pattern of processing is called a traversal.
- String Traversal can be done with while or for loops.

Eg: Using while loop

```
s="Hello"
i=0
while i<len(s)
        print s[i]
        i=i+1
```

Using for loop

```
for i in range (0,len(s)):
        print  s[i]
```

len (s) = no.of characters in s= 5

i varies from 0 to 4

S[i] means each character in the string

S[0]= H s[1]= e etc

Another Method Using for loop(Easy method)

```
i)for i in s:
        print i
```

Here i represents each character in the string s

DOWNLOADED FROM KTUNOTES.IN

STRING OPERATORS

1.STRING CONCATENATION

- **+ operator is used**

Eg:

```
s1="Hello"
s2="World"
s3=s1+s2
print s3
```

2.STRING COMPARISON

- **==, != , <, <= , > , >= operators are used**

Eg:

```
s1=raw_input()
s2=raw_input()
if (s1==s2):
    print "Equal"
elif(s1<s2):
    print "s1 precedes s2"
else:
    print"s1 succeeds s2"
```

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

3.STRING REPLICATION USING * OPERATOR

- **Used to repeat a string for a given no.of times**

```
s=“Hello ”
s2=s*3
print s2
```

Output : HelloHelloHello

4.IN AND NOT IN OPERATORS

- **Used to check whether a character occurs in a string**

```
s=“Hello ”
if(‘e’ in s):
        print (“Present”)
else:
        print “Not Present”
```

Output : Present

```
s=“Hello ”
        if(‘e’ not in s):
                print (“Not Present”)
        else:
                print “Present”
```

Output : Present

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

- **Python does not handle upper- and lowercase letters. All the uppercase letters come before all the lowercase letters.**

- **Strings are Immutable. Ie we can't change the characters in a string. But we can delete the entire string**

Eg:

```
greeting = "Hello, world!"
greeting[0] = 'J'        # ERROR!
print   greeting
```

Deleting an entire string

Syntax: del string name

```
s="Hello"
del  s
```


DOWNLOADED FROM KTUNOTES.IN

STRING SLICES

- A segment(part) of a string is called a slice
- Slice operator (:) is used to extract a part (slice) of a string

Syntax: string name[n : m : stepsize]

returns the part of the string from the nth character to the mth character, including the first character but excluding the last

Eg: s= "Program"

-8	-7	-6	-5	-4	-3	-2	-1
0	1	2	3	4	5	6	7
P	r	o	g	r	a	m	s

1.s[0:5:1] - Progr

2.s[3:7:1] -gram

3.s[2:6:2] - or

4.s[1:4] - rog

5.s[-2:-7:-1] -margo

6.s[-5:-2] -gra

If step size is not given default value = 1

DOWNLOADED FROM KTUNOTES.IN

➢**If you omit the first index (before the colon), the slice starts at the beginning of the string. If you omit the second index, the slice goes to the end of the string.**

1.s[:5:1] - Progr

2.s[:4:] OR s[:4] - Prog

3.s[2::2] - orm

4.s[2::] OR s[2:] - ograms

5.s[::] OR s[:] OR s[::1] - Programs

6.s[::-1] - smargorP

.

-8	-7	-6	-5	-4	-3	-2	-1
0	1	2	3	4	5	6	7
p	r	o	g	r	a	m	s

DOWNLOADED FROM KTUNOTES.IN

STRING FUNCTIONS(METHODS)

1.find() method

- **Used to find the index of a given character in the string. If character is not present the function returns -1**

  Syntax: string.find(character)

0	1	2	3	4	5
P	y	t	h	o	n

Eg:

```
s="Python"
n1=s.find('t')  - Output : 2
n2=s.find('k')  - Output : -1
```

- **Also used to search whether a substring is present in a given string. If substring is present it returns the starting index of substring.If not present the function returns -1**

```
s="Python"
n1=s.find('tho') - Output : 2  ( starting index of substring)
n2=s.find('the') - Output : -1 (substring not present)
```

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

2.count() method

- **Used to find the no.of occurrences of a substring or character present in a given string. If substring or character is not present the function returns -1**

  s="substring search in a string"

Syntax: string.find(substring/character)

n1=s.count('in') - Output : 3

n2=s.count('ing') - Output : 2

n3=s.count("search") - Output :1

n4=s.count("t") – Output : 2

3.replace() method

- **Used to return a copy of a string in which a given substring or character is replaced with another.**

Syntax: string.replace(substring,newsubstring)

string.replace(character, new character)

Eg1 s="Hello good morning"

s2=s.replace("morning","evening") - s2: Hello good evening

Eg2 s="Hello good morning"

s2=s.replace(" ", "*") - s2: Hello*good *morning (Here space is replaced with *)

DOWNLOADED FROM KTUNOTES.IN

4.Split() method

- **Used to split a string based on a delimiter and returs a list of words**

Syntax: s.split(delimiter)

s= “hello World”

s2=s.split(“ “) → String splits at space - s2: [‘hello’ , ‘World’] (s2 - list of words)

s= “hello, How Are U”

s2=s.split(“ , “) → String splits at , - s2: [‘hello’ , ‘How Are U’]

5.join() method

- **Used to join a list of words by using a delimiter (Eg: hyphen -) and returns a string**

Syntax: delimiter.join(list of words)

s2=[‘hello’ , ‘World’]

s3= - . join(s2)

Then s3: hello-World (The words are joined by using – and stored in s3)

DOWNLOADED FROM KTUNOTES.IN

6.upper() method

➢Used to produce an uppercase copy of the string .

Syntax: string.upper()

s=“Hello”

s2=s.upper() - s2 : HELLO

7.lower() method

➢Used to produce a lowercase copy of the string .

Syntax: string.lower()

s=“HELLO”

s2=s.lower() - s2: hello

8.capitalize() method

➢ Used to produce a copy of the original string and converts the first character of the string to a capital (uppercase) letter and converts all other characters in to lowercase letters

Eg: s=“hello how are U”

s2=s.capitalize() - s2: Hello how are u

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

9.min() method

➢Used to return the minimum alphabetical character from a string .

Syntax: min(string)

s="Hello"

s2=min(s) - s2: H (In Python All the uppercase letters come before all the lowercase letters

10.max() method

➢Used to return the maximum alphabetical character from a string .

Syntax: max(string)

s="Hello"

s2=max(s) - s2: o

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

STRINGS-UNIVERSITY QUESTIONS

2019 SCHEME UNIV QNS

3 Write a program that converts a sentence entered by the user into a list of words (3)

Ans)

```
x = raw_input("Enter a sentence:")
a= x.split(" ")
print  a
```

13 4 Write a Python program to add 'ing' at the end of a string. If the string already ends with 'ing' then add 'ly' (8)

```
str1 = raw_input("Enter a string:")
y=str1[-3: :]
if(y != "ing" ):
   str2=str1 + "ing"
else:
   str2=str1 + "ly"
```

Stores the last 3 characters of the string in 'y'

Ktunotes.in

(b) Write a Python program to reverse a string and print whether its palindrome or not. (8)

Ans)

```
s= raw_input("Enter a string")
s2=s[::-1]
print "Reverse:",s2
if(s==s2):
    print "Palindrome"
else:
    print "Not Pal"
```

s[::-1] means last character to first character ie reverse

DOWNLOADED FROM KTUNOTES.IN

(b) Write a program to
i) convert all small letters in a string into capital letters
ii) find the occurrence of a given substring. (7)

Ans)

```
s= raw_input("Enter a string")
s2=s.upper()
print "Uppercase string:" ,s2
x= raw_input("Enter a substring")
k=s.count(x)

if(k==-1):
    print"Substring not found"
else:
    print "No.of occurrences:", k
```

3. Write a Python code to check whether two strings are equal or not.

Ans)

```
s1= raw_input("Enter first string")
s1= raw_input("Enter second string")
if(s1==s2):
   print "Equal"
else:
   print "Not"
```

Q.Let s='Hello World' be a string. What will be the output of following expressions?

i) s[0:7] ii) s[-3] iii) s[::-1] iv) len(s) v) s[8:0:-2] vi) s[2::]

Ans) i) Hello Wo ii) r iii) dlroW olleH iv) 11 v) rWol vi) llo World

DOWNLOADED FROM KTUNOTES.IN

Q

(b) Write a Python program to count number of vowels, consonants, words and questionmarks in a given string. (6)

Consonents -> Non vowels
alphabets other than a,e,i,o,u

```
Ans) s= raw_input("Enter a string")
     c1=c2=c3=c4=0
     s1=s.lower()
     for  k in s1:
          if(k=='a' or k=='e' or k=='i' or k=='o' or k=
                    c1=c1+1
          elif( (k>='a' and  k<='z' ) ):
                    c2=c2+1
          elif(k==' ' or k=='\t'):
                    c3=c3+1
          elif(k=='?'):
                    c4=c4+1
     print  "No.of Vowels :" c1
     print  "No.of consonents:" , c2
     print   "No.of words:" ,c3
     print  "No.of qn  marks:",c4
```

Store s in s1 after converting in to same case (here lowercase)
Otherwise while checking for vowels & consonents we need to check uppercase and lowercase letters separately

This condition is checked only if the condition in the if part is false .ie character is not a vowel.So here we Check whether character is a non vowel alphabet(consonant)

This condition is checked only if the condition in the if part and first elif part are false.
.ie character is not a vowel or consonant.So here we Check whether the character is a single space or tab space. If so word count is incremented . Because here we assume that words are separated by single space or tab space

This condition is checked only if first 3 conditions are false.

Q

12 str1 ="Jython" (2)
str1[0] = "P" Is there any error in this Python code? Justify your answer.

Ans). Yes .string is immutable. we cant change individual characters of string. So there is error in the code. we can't store the character 'P' at index 0 by replacing 'J'

DOWNLOADED FROM KTUNOTES.IN

2015 SCHEME REPEATED UNIV QNS

Q 13. Write a program to remove all vowels from a given string. (3)

Ans)

```
s= raw_input("Enter a string")
for k in s:
   if(k=='a' or k=='e' or k=='o'):
         s2=s.replace(k,"")
 print s2
```

Since string is immutable we cant change original string s. Here we store s in s2 by replacing vowels with null string

Q 12. Consider the following code snippet:

```
one = "This a test!"
one[2] = "u"
print one
```

What error will the above code produce? Why is the error caused? Write the Python code which will work around this issue and print the string "Thus a test!" as output. (3)

Ktunotes.in

Ans) Type Error. Because strings are Immutable and we cant modify the individual characters

```
one = "This a test"
s2=s.replace('i','u')
print s2
```

Since string is immutable we cant change original string s. Here we store s in s2 by replacing 'i' with 'u'

Q 23. a) Write a program to replace a substring with a new substring in the given string. (7)

Ans)

```
s= raw_input("Enter a string")
x= raw_input("Enter a substring")
y= raw_input("Enter another substring")
s2=s.replace(x,y)
print s2
```

Here s is stored in s2 by replacing the substring x with y

DOWNLOADED FROM KTUNOTES.IN

Q Suppose S="*mary had a little lamb*". Write the python code to (2)
i) Replace "*lamb*" with "*kid*"
ii) Find starting index of substring "*had*" in the string S .

Ans)

```
s= "mary had a little lamb"
s2=s.replace("lamb","kid")
k=s.find("had")
print  k
```

Q 11. Let fruit='apples' be a string. What will be the output of the following expressions:
i) len(s1) (ii) s1[0:4] (iii) s1[6] (iv) s1[-4]
12. Let Data represent the list ['circle','square','triangle']. Write the expressions for tol operations:

Ans) i) 6 ii) appl iii)Error-String index out of range iv) p

Ktunotes.in

Q Write a Python program to traverse a string in reverse order. (3)

```
s= raw_input("Enter a string")
for i in range(len(s)-1 , -1, -1):
   print s[i],
```

Let s="Hai"
last index: 2 (ie len(s)-1)
first index =0
i varies from len(s) to 0 for traverse in reverse order .
So...for i in range(len(s)-1,-1,-1)
If we need to get index 0 then final value should be -1.
Step size should be decreased by 1.

Q b) Describe any four string methods in python with example. (4)

Refer Note

DOWNLOADED FROM KTUNOTES.IN

Q 9 Let str='String operations in python'. What will be the output of the following expressions: (2)

i. str[: -2] ii. str[:] iii. str[10 :] iv. str[-6 : -2]

10 Write a Python program to read a string and use appropriate string operation to print the string 20 times in a line with no space. Do not use any iterative statement. (3)

11 Write a Python program to read a string and find the number of vowels and consonants in the string. (3)

9 Ans) **i) String operations in pyth** **ii) String operations in python**

iii) rations in python **iv) pyth**

10 Ans)
```
s= raw_input("Enter a string")
s2=s*20
print s2
```

11 Ans) Refer 2nd qn

Ktunotes.in

Q 10. Let s='orange' be a string. What will be the output of the following expressions.

(i) (a) s[0:4] (b) s[6] (c) s[-4] (d) s[-3:]

(ii) You are given a string, for instance, 'How are you'. Split the string on to a " " (space) delimiter and join using a – (hyphen) delimiter. (final output of the given example will be 'How-are-you')

(3)

i) a) oran **b) Error-String index out of range** **c) a** **d) naro**

ii) s= 'How are you'
```
x=s.split(" ")
s2='-' . join(x)
print s2
```

Here, x= ['How' , 'are' , 'you']
Next, These words are joined by using – and stored in s2

DOWNLOADED FROM KTUNOTES.IN

2015 SCHEME OTHER QNS

Q

20 a) Write a menu driven Python program to read a string and perform the following string operations:

(i) Slice the string to two separate strings; one with all the characters in the odd indices and one with all characters in even indices.

(ii) Replace all the spaces in the input string with * or if no spaces found, put $ at the start and end of the string. (7)

Ans)

```
s= raw_input("Enter a string")
print "Menu \n 1.Slice \n 2.Replace"
n=input("Enter choice")
if(n==1):
    s1=s[0::2]
    s2=s[1::2]
    print s1
    print s2
elif(n==2):
    c=0
    for k in s:
        if(k==" "):
            c=c+1
    if(c==0):
        s2='$'+s+'$'
    else:
        s2=s.replace(" ","*")
    print s2
```

Here, s[0::2] means characters at even indices and s[1::2] contains characters at odd indices

- Here, first we check whether the string contains spaces(space count is stored in c)
- if no.of spaces in string s =0 it is stored in another string s2 by adding a '$' character at the start and end by using + operator
- If string s contains spaces it is stored in another string s2 by replacing spaces with

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

Q

Write a Python program to reverse the first half and second half of an even lengthed string separately. (Eg: if the string is "gooddays output should be "doogsyad") (3)

11 Write a Python program to remove the characters at odd index positions in a string. (3)

10Ans)

```
s= raw_input("Enter a string(even length)")
n=len(s)
s1=s[0:n/2]
s2=s[n/2:]
s3=s1[::-1]+s2[::-1]
print s3
```

n= string length(no.of caharacters)
for an even lengthed string s the index of first half will be from 0 to n/2 -1
index of 2nd half will be from n/2 to n-1
For eg. if length n= 6
frst half: 0 to 2
sec half : 3 to 5
s1 stores first half and s2 stores 2nd half reverse of s1 and s2 are combined and stored in s3

11 Ans)

```
s= raw_input("Enter a string")
s1= s[0::2]
print s1
```

The result of s[0::2] is the characters at even indices.

Ktunotes.in

Q

21 a) Write a Python program to input a string and perform the following operations.

i. Reverse the string without using reverse() function.

ii. Check for a substring in the string

iii. Find all the occurrences of a particular character in the string and print the indices at which the character appears. (7)

Ans i)

```
s= raw_input("Enter a string")
s2=s[::-1]
print "Reverse", s2
```

ii)

```
s= raw_input("Enter a string")
x= raw_input("Enter a sub string")
k=s.find(x)
if(k==-1):
    print"Substring Not Present"
else:
    print "Present"
```

iii)

```
s= raw_input("Enter a string")
x= raw_input("Enter a character")
for i in range(0,len(s)-1,1):
    if(s[i]==x):
        print "Found at index :", i
c= s.count(x)
print "No.of occurrences:",c
```

DOWNLOADED FROM KTUNOTES.IN

Q 22. (a) Write a program to check if a given string is a palindrome or not, without reversing the original string. (7)

```
s= raw_input("Enter a string")
n=len(s)
f=0
for i in range(0,n/2,1):
      if(s[i]!= s[n-i-1]):
          f=1
          break
if(f==0):
   print "Palindrome"
else:
   print "Not"
```

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

LISTS

- A list is an ordered set of values, where each value is identified by an index.
- The values that make up a list are called its elements.
- Individual elements can be of any type, even another list.
- List is MUTABLE .Because we can insert ,delete or modify the elements in a list

CREATING A LIST(Direct method)

- A list created by enclosing the all the elements inside square brackets ([ ]) separated by commas

list name=[element1,element2,…….]

Example: a= [10, 20, 30, 40]

s=[“Python", “Java", “C"]

list1= ["hello", 2.0, 5, [10, 20]]

- The last list contains a string, a float, an integer, and another list
- If a list contains another list it is known as nested list. Here list1 is a nested list.

DOWNLOADED FROM KTUNOTES.IN

- **There is a special list that contains no elements. It is called the empty list.**

Eg: b = []

CREATING A LIST BY READING ELEMENTS FROM KEYBOARD

```
x = []
n=input("enter the no.of elements")
print  "\n Enter the Elements"
for i in range(1,n+1):
        k=input()
x. append(k)
```

Inside the for loop 'n' numbers are entered and appended to the list

DISPLAYING A LIST

Syntax: print listname

Eg: x =[1,2,3,4,5]

```
print x
```

Output: [1,2,3,4,5]

DOWNLOADED FROM KTUNOTES.IN

ACCESSING LIST ELEMENTS

- **The elemets are accessed by using index numbers. The index start at 0.**
- **The index of first element is 0,second element is 1,third element is 2 and so on.**
- **If list contains ‘n’ elements index number varies from 0 to (n-1)**
- **Python allows negative indexes for list elements. The negative index starts from -1(last element)**
- **The negative index of last element is -1,second last element is -2, and so on.**

Eg1: a=[2,4,8,12]

-ve index	-4	-3	-2	-1
+ve Index	0	1	2	3
Element	2	4	8	12

Print a[2] **Output: 8**

Print a[-3] **Output :4**

DOWNLOADED FROM KTUNOTES.IN

Eg2: list1=[“Physics”,2016,90.5]

Print list1[2] **Output: 90.5**

Print list1[0] **Output :Physics**

Eg3: list1=[“Happy” , 20 , [2,10,1,5]]

Print list1[2] **Output: [2,10,1,5]**

Print list1[1] **Output :20**

Print list1[2] [1] **Output:10**

Print list1[1] **Output :20**

Print list1[0][1] **Output :a**

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

LIST LENGTH

- **The function len is used to find the length of a list(number of elements in the list)**

Syntax: len(list name)

Eg1: num=[2,4,6,8]

l=len(num) l=4

Eg2: list1=[“Happy” , 20 , [2,10,1,5]]

l=len(list1) l=3

Ktunotes.in

LIST TRAVERSAL

- **A lot of computations involve processing a list one element at a time.**
- **We may start at the beginning, select each element in turn, do something to it, and continue until the end. This pattern of processing is called a traversal.**
- **List Traversal can be done with while or for loops.**

DOWNLOADED FROM KTUNOTES.IN

Eg: a=[1,2,3,4]

Using while loop

```
i=0
    while i<len(a)
        print a[i]
        i=i+1
```

len (a) = no.of elements in a= 4

i varies from 0 to 3

a[i] means each element in the list a

a[0]= 1 a[1]= 2 etc

Using for loop

```
for i in range (0,len(a)):
    print  a[i]
```

Another Method Using for loop(Easy method)

```
for i in a:
    print i
```

Here i represents each element in the list a

Ie i=1 print1

i=2 print 2 etc

Output: 1
2
3
4

DOWNLOADED FROM KTUNOTES.IN

Eg: s=[“Physics” ,”Chemistry”,”Maths”]

```
for i in s:
    print i
```

Output: Physics

Chemistry

Maths

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

LIST SLICING

- A slice is a subset of a list .
- Slice operator (:) is used to extract a slice of the list

Syntax: list name[n : m : stepsize]

returns the subset of the list from the nth element(including) to the mth element(excluding)

Eg: s=[1,2,3,4,5,6,7,8]

1.s[0:5:1] Output: [1,2,3,4,5]

2.s[3:7:1] Output:[4,5,6,7]

3.s[2:6:2] Output:[3, 5]

4.s[1:4] Output:[2,3,4]

5.s[-2:-7:-1] Output:[7,6,5,4,3]

6.s[-5:-2] Output :[4,5,6]

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

- **If you omit the first index (before the colon), the slice starts from beginning of the list. If you omit the second index, the slice goes to the end of the list.(+ve step size)**
- **If you omit the first index (before the colon), the slice starts from end of the list. If you omit the second index, the slice goes to the beginning of the list.(-ve step size)**

s=[“Hello” , 11 , 5 , 3 , 4.5 , ’x’ ,10]

1.s[:5:1] : [“hello”,11,5,3,4.5]

2.s[:4:] OR s[:4] : [“hello”,11,5,3]

3.s[2::2] : [5,4.5,10]

4.s[2::] OR s[2:] : [5 , 3 , 4.5 , ’x’ ,10]

5.s[2::-1] : [5,11,,”Hello”]

5.s[::] OR s[::1] : [“Hello” , 11 , 5 , 3 , 4.5 , ’x’ ,10]

6.s[::-1] : [10, ’ x’, 4.5, 3, 5, 11, ”Hello”]

DOWNLOADED FROM KTUNOTES.IN

DELETING LIST ELEMENTS & ENTIRE LIST

- **del statement can be used if we know exactly which element is deleting.**

Eg: list1=[2,5,3,8,9]

del list1[1] will delete the element at index 1

list1 becomes:[2,3,8,9]

DELETING AN ENTIRE LIST

list1=[2,5,3,8,9]

del list1 will delete the entire list.

DELETING A PART OF THE LIST

list1=[2,5,3,8,9]

del list1[1:3] will delete the elements at index1 and index2

list1 becomes: [2,8,9]

DOWNLOADED FROM KTUNOTES.IN

LIST OPERATIONS

- **There are 4 basic list operations in Python**

1. concatenation 2.Repetition 3. in 4. not in

1. concatenation (+)

- **Joins two lists**

Eg: list1=[2,5,3,8,9]

list2=[3,10,20]

list3=list1+list2 list3= [2,5,3,8,9,3,10,20]

The elements in list1 and list2 are combined and stored in list3

2.Repetition (*)

- **This operator replicates the elements in the list**

Eg: list1=[2,5,8]

list2=3*list1 list2=[2,5,8,2,5,8,2,5,8]

The elements in list1 are replicated 3 times and stored in list2

DOWNLOADED FROM KTUNOTES.IN

3. in

- Used to check whether an element present in a list.
- Returns true if element is present .Otherwise returns false.

```
Eg: list1=[2,5,3,8,9]
    if(2 in list1):
        print ”Present”
    else:
        print”Not Present”
```

4. not in

- Used to check whether an element not present in a list.
- Returns true if element is not present .Otherwise returns false.

```
Eg: list1=[2,5,3,8,9]
    if(2 not in list1):
        print ”Not Present”
    else:
        print”Present”
```


DOWNLOADED FROM KTUNOTES.IN

BUILT IN LIST METHODS (FUNCTIONS)

append()

- **This method is used to add an element to the end of a list**

  Syntax: listname.append(element)

  Eg: a=[1,2,3,4]

  a.append(90)

  Now a becomes [1,2,3,4,90]

extend()

- **This method is used to add all the elements of a sequence at the end of a list**

  Syntax: listname.extend(sequence)

  Eg: a=[1,2,3,4] b=[5,6,7]

  a.extend(b)

  Now a becomes [1,2,3,4,5,6,7]

DOWNLOADED FROM KTUNOTES.IN

insert()

- This method is used to insert an at a specified index in a list

Syntax: listname.insert(index,element)

Eg: a=[“C”, ”Python”   ,”Java”]

a.insert(1,”C++”)

Now a becomes = [“C”, “C++” , ”Python”   ,”Java”]

NOTE: Suppose we want to replace an element with another element,

Eg: list1=[2,5,3,8,9]

We want to replace element at index position 2 (ie 3) with 300.

list1[2]=300. Now list1=[2,5,300,8,9]

remove()

- This method is used to remove a given element from a list

Syntax: listname. remove(element)

Eg: a=[“C”, ”Python”   ,”Java”]

a.remove(“Python”)

Now a becomes : [“C” ,”Java”]

DOWNLOADED FROM KTUNOTES.IN

pop()

- This method is used to remove an element from a given index from a list
- If no index is given, it removes the last element.

Syntax: listname. pop(index)

Eg: a=[“C”, ”Python” ,”Java”]

b=a.pop(0) b:”C”

c=a.pop() c:”Java”

NOTE: Another way to delete an element at an index

- **del statement can be used to delete an element at an index**

Eg: list1=[2,5,3,8,9]

del list1[1] will delete the element at index 1

list1 becomes:[2,3,8,9]

DOWNLOADED FROM KTUNOTES.IN

index()

- This method is used to find the lowest index of an element in a list

Syntax: listname. index(element)

Eg: a=[1,2,3,4,3,5,1,9,1]

b= a.index(1) . Then b=0

count()

- This method is used to count the no.of occurences of an element in a list

Syntax: listname. count(element)

Eg: a=[1,2,3,4,3,5,1,9,1]

b= a.count(1) . Then, b=3

reverse()

- This method is used to reverses the elements in a list

Syntax: listname. reverse()

Eg: a=[“C”, ”Python” ,”Java”]

a.reverse()

Now a becomes = [“Java”, ”Python” , ”C”]

DOWNLOADED FROM KTUNOTES.IN

sort()

- **This method is used to sort the elements in a list**

  Syntax: listname. sort()

  Eg: a=[“C”, ”Python” ,”Java”]

  a.sort() **Now a becomes = [“C”, ”Java” , ”Python”]**

- **If we need to sort in descending order:**

  Syntax: listname. sort(reverse=True)

  Eg: a=[“C”, ”Python” ,”Java”]

  a.sort(reverse=True) **Now a becomes = [“Python”, ”Java” , ”C”]**

Sorted()

- **This method is used to store the sorted elements of a list in another list**

  Syntax: list2=sorted(list1)

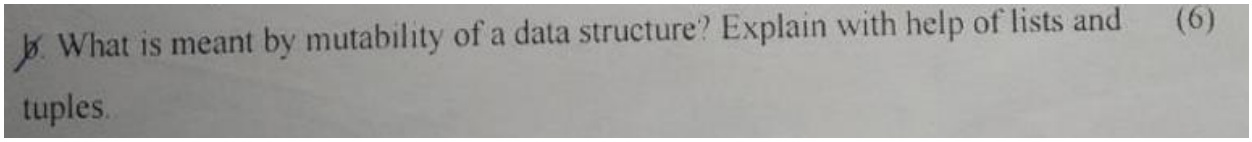
  Eg: a=[“C”, ”Python” ,”Java”]

  b=sorted(a) **b= [“C”, ”Java” , ”Python”]**

DOWNLOADED FROM KTUNOTES.IN

MODULE2-LIST-UNIVERSITY QUESTIONS

2019 SCHEME UNIVQ


What is meant by mutability of a data structure? Explain with help of lists and tuples. (6)

Mutable structure : we can add new elements, delete elements or modify theelements . eg: list , dictionary

Immutable structure : We can't add, delete or modify elemnts in the datastructure . Eg: string , number

A list is an ordered set of values,where each value is identified by an index.

Values are enclosed within square brackets

In a list we can add,delete or modify elements. So it is mutable

Eg: x=[10,20,30]

x.insert(1,100) → insert an element 100 at index 1

Now list becomes : x=[10,100,20,30]

del x[2] → remove element at index 2.

Now list becomes : x=[10,100,30]

x[2] = 90 → replace element at index 2 with 90 Now, x=[10,100,90]

Also we can apply the operations sort, reverse, etc

A tuple is also an ordered set of values, where each value is identified by an index.

Values are enclosed within parenthesis

But We can't add ,delete or modify elements in a tuple. So it is immutable.

13 Differentiate between tuple and list. (3)

Ans .

List is an ordered collection of elements.Tuple is also an ordered collection of elements.

Elements can be: numbers , strings, other lists or tuples etc

Difference

1. List is mutable.ie we can change or delete individual elements
   Tuple is immutable. individual elements can't be modified or deleted

DOWNLOADED FROM KTUNOTES.IN

2.List uses [] to enclose elements. Tuple uses ( ) to enclose individual elements.

3.Since tuples are immutable iteration over tuple is faster.

14 a. Write a program to print index at which a particular value exists in a list of numbers. If the value exists at multiple locations in the list, then print all the indices. (8)

```
x =[]
f=0
n=input("enter the no.of elements")
print "\nEnter the Elements"
for i in range(1,n+1):
        k=input()
    x.append(k)
print    "List is:" ,x
s=input("Enter element to be searched")
for i in range (0,len(x)):
    if (x[i]==s):
        print "Found at index:" ,i
        f=1
if(f==0):
    print "Not Found"
```

The'n' elements are read one by one in k and each element is appended to the list x

elements of the list are accessed by using index numbers. index number varies from 0 to listlength-1

4. Write a Python code to search an element in a list.

```
x =[]
f=0
n=input("enter the no.of elements")
print "\n Enter the Elements"
for i in range(1,n+1):
        k=input()
        x.append(k)
print  "List is:" ,x
s=input("Enter element to be searched")
if s in x:
    print "Found"
else:
   print "Not Found"
```

we check whether the search value is present in the list using in operator.

DOWNLOADED FROM KTUNOTES.IN

(c) Write a Python program to input a list of n numbers. Calculate and display the average of numbers. Also display the cube of each value in the list. (5)

```
x =[]
sum=0
n=input("enter the no.of elements")
print "\nEnter the Elements"
for i in range(1,n+1):
        k=input()
        x.append(k)
print "List is:" ,x
for a in x:
        s=a*a*a
        print  s
        sum=sum+a
print sum
av=(float)sum/n
print "Average:",av
```

Ktunotes.in

Write a Python program to input a list of n numbers. Calculate and display the sum of cubes of each value in the list. (5)

```
x =[]
sum=0
n=input("enter the no.of elements")
print "\nEnter the Elements"
for i in range(1,n+1):
        k=input()
        x.append(k)
print "List is:" ,x
for a in x:
        s=a*a*a
        print  s
        sum=sum+s
print sum
```

Here 'a' stores the elements of the list one by one.. cube of each value is calculated ad stored in s. sum of cubes are stored in the variable sum.

DOWNLOADED FROM KTUNOTES.IN

Write a Python program to input a list of N numbers. Display the sum of even numbers and odd numbers in the list.

```
x =[]
s1=s2=0
n=input("enter the no.of elements")
print "\nEnter the Elements"
for i in range(1,n+1):
        k=input()
        x.append(k)
print "List is:" ,x
for a in x:
        if(a%2==0):
                s1=s1+a
        else:
                s2=s2+a
print "Sum of even nos:",s1
print "Sum of odd nos:",s2
```

Here 'a' stores the elements of the list one by one.. If the element is even it is added to s1 otherwise added to s2

22. a) With examples explain the built-in methods used for list operations. (7)

Refer note

Ktunotes.in

iii) Literals (6)

13. (a) Write a Python code to add two matrices using list. (6)

```
m=input("Enter no.of rows: ")
n=input("Enter no.of columns: ")
matrix1=[]
matrix2=[]
matrix3=[]

print "\nEntering the Elements for matrix1....."
for i in range(0,m):
        a=[]
        for j in range(0,n):
                k=input()
                a.append(k)
        matrix1.append(a)
```

DOWNLOADED FROM KTUNOTES.IN

```
print "\nEntering the Elements for matrix2....."
for i in range(0,m):
        a=[]
        for j in range(0,n):
                k=input()
                a.append(k)
        matrix2.append(a)

for i in range(0,m):
        a=[]
        for j in range(0,n):
                k=matrix1[i][j] + matrix2[i][j]
                a.append(k)
        matrix3.append(a)


for k in matrix3:
        print k
```

- matrix1 is a list in which individual elements are lists that represent rows of that matrix.

Eg: matrix1= [[1,2] , [5,3]]
matrix 1 is a 2x2 matrix
1st row: [1,2]
2nd row:[5,3]

- Here list 'a' is used to store each row of matrix1.
- each row contains n elements.
- within inner loop the elements of a row are entered from keyboard and append to list 'a'.
- Then that row (ie list a) is appended to matrix1
- Same method for matrix2

- matrix3 is a list in which individual elements are lists that represent rows of that matrix.
- Here list 'a' is used to store each row of matrix3.
- each row contains n elements.
- within inner loop the elements of a row are calculated as the sum of elements in the same row of matrix1 and matrix2 and the resultant elements are append to list 'a'.

Eg. row 2 of matrix 3= sum of corresponding elements of row2 of matrix1 and matrix2

- Then that row (ie list a) is appended to matrix3

program to subtract two matrices using list.

k=matrix1[i][j] – matrix2[i][j]

All other steps -same

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

2015 SCHEME UNIVQ

23. (a) Write a python code to find transpose of a matrix using list.

```
m=input("Enter no.of rows: ")
n=input("Enter no.of columns: ")
matrix1= matrix2 =[]
print "\nEntering the Elements for matrix1....."
for i in range(0,m):
        a=[]
        for j in range(0,n):
                k=input()
                a.append(k)
        matrix1.append(a)

print "\nCalculating Transpose"
for i in range(0,n):
        a=[]
        for j in range(0,m):
                a.append(matrix1[j][i])
        matrix2.append(a)

for k in matrix2:
    print k
```

- Transpose is stored in matrix2
- if order of matrix1 is mxn then order of transpose matrix will be n x m that's why outer loop varies from 0 to n-1 and inner loop varies from 0 to m-1
- (i,j) th element of matrix2 = (j,i)th element of matrix1
- a is used to store each row of matrix2
- In the inner loop elements of each row is calculated and appended to list a.
- Then that row is appended to matrix2

Ktunotes.in

24. a) Write a program to sort a list of names in alphabetical order and print the sorted list in uppercase. (7)

```
x =[]
n=input("How many names?")
print "\nEnter the names"
for i in range(1,n+1):
        k=raw_input()
        x.append(k)
print "List is:" ,x
x.sort()
print "Sorted List:",x
for a in x:
        b=a.upper()
print b
```

The names in the list are sorted

Here 'a' stores the names in the list one by one..

Then the names are stored in b after converting in to uppercase .

Then these names are displayed

DOWNLOADED FROM KTUNOTES.IN

22 a) Write a Python program to create a list of strings. Separate palindromes in the list to another list without using *reverse( )* function. (7)

```
x =x2=[]
n=input("How many names?")
print "\nEnter the names"
for i in range(1,n+1):
        k=raw_input()
        x.append(k)
print "List is:" ,x
for a in x:
        b=a[::-1]
        if(a==b):
                x2.append(a)
print x2
```

Here 'a' stores the names in the list one by one..

Then the names are stored in b after reversing it. .

Then the name and it's reverse are compared .

If palindrome, that name is added to list

Ktunotes.in

Q2 Let data=[23,56,67,2,[6,7,12],123]. Write the expressions for following operations in Python: (3)

i) Replace the value 67 with 89
ii) Print the value 12.
iii) Remove the value 56 from list.

```
i)    data[2] = 89
ii)   print data[4][2]
iii)  data.remove(56)
```

Q3 Let Data represent the list ['circle','square','triangle']. Write the expressions for following operations:

i) Replace the value 'circle' with 'ellipse'
ii) Add a new value 'rectangle' top end of list
iii) Remove the values 'square' and 'triangle' from list.

DOWNLOADED FROM KTUNOTES.IN

i) data[0] = 'ellipse'

ii) data.append('rectangle')

iii) data.remove('square') data.remove('triangle')

Q4 Consider the list scores = [5, 4, 7, 3, 6, 2, 1] and write the Python code to perform the following operations: (3)

i) Insert an element 9 at the beginning of the list.
ii) Insert an element 8 at the index position 3 of the list.
iii) Delete an element at the index position 4.

i) scores.insert(0,9) ii) scores.insert(3,9) iii) del scores[4] Or scores.pop(4)

9 What will be the output of the given code? (2)

```
>>>values = "Apple, orange, grapes"
>>>list = values.split(",")
>>>print("list:", list)
```

Ktunotes.in

Output :

list : ['Apple' , 'orange' , 'grapes']

DOWNLOADED FROM KTUNOTES.IN

DICTIONARIES

- A Dictionary is an unordered collection of items.
- Each item has a key and a corresponding value that is expressed as a pair (key: value).
- The (key, value) pairs can be written in any order.
- Keys are unique within a Dictionary while values may not be unique.
- The values can be of any type but the keys must be of an immutable datatype such as numbers, strings or tuples.

Ktunotes.in

- Dictionary is mutable .We can add new items or change the values of existing items

DICTIONARY OPERATIONS

1. Dictionary Creation
2. Displaying dictionary
3. Dictionary Traversal
4. Updating dictionary elements
5. Deletion of Dictionary elements
6. Membership operations

DOWNLOADED FROM KTUNOTES.IN

DICTIONARY OPERATIONS

CREATING A DICTIONARY

- **A Dictionary is created by enclosing the all the items inside curly brackets ({}) separated by commas**

Syntax : Dictionary name={ key1 : value1 , key2 : value2 , key3 : value3 ,…….}

Eg: weekdays= {1: “Sunday” , 2: “Monday “ , 3: “Tuesday “, 4: “Wednesday”}

student = { “RollNo” : 10 , “Name” : “John” , “mark” : 280 }

color= {5: “Red” , 2: “Blue “ , 3: “Green “, 4: “Orange”}

Employee = {“Name” : “James” , “Designation” : “Programmer” , “Salary” : 60000 }

Dict1 = { } → empty Dictionary

DOWNLOADED FROM KTUNOTES.IN

CREATING A DICTIONARY BY READING KEYS AND VALUES FROM KEYBOARD

```
x = { }
n=input("enter the no.of elements in the dictionary")
print "\nEnter the Elements(key, value)"
for i in range(1,n+1):
        k=input()
        v=input()
        x[k] = v
print "Dictionary is:" , x
```

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

ACCESSING DICTIONARY ITEMS

- The elemets are accessed by using keys.
- Keys can be used either inside square brackets [] or with the get() method.
- In the first method if the key is not found an error is displayed. With get function, If the key is not found it returns None.

Syntax : dictionary name [key]

Eg: student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }

print student["RollNo"] will display 10

n= student["Name"] value of n will be John

DOWNLOADED FROM KTUNOTES.IN

DISPLAYING A DICTIONARY

print dictionary name

Eg: print student

DICTIONARY LENGTH

- **The function len is used to find the length of a Dictionary(number of items in the dictionary)**

  Syntax: len(Dictionary name)

Eg: student = { “RollNo” : 10 , “Name” : “John” , “mark” : 280 }

l=len(student) l=3

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

DICTIONARY TRAVERSAL

- **Dictionary Traversal can be done with for loops.**

student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }

```
for i in student:
    print i
```

Here 'i' represents each key in the dictionary.
So here keys are displayed one by one

Output: RollNo
Name
mark

```
for i in student:
    print student[i]
```

Here 'i' represents each key in the dictionary.
So here values are displayed one by one

Output: 10
John
280

```
for i in student:
    print i, ":" ,student[i]
```

Here 'i' represents each key in the dictionary.
So here keys and values are displayed one by one

Output: RollNo : 10
Name : John
mark : 280

DOWNLOADED FROM KTUNOTES.IN

ADDING NEW ELEMENTS IN TO DICTIONARY

- **Dictionaries are mutable. We can add new items or change the value of existing items using an assignment operator.**
- **We can add a new item (key-value pair) by using the syntax:**

dictionary[key] =value

Eg: **student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }**

student["Grade"]="A"

Now,student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 , "Grade" : "A"}

Ktunotes.in

MODIFYING ELEMENTS IN A DICTIONARY

- **We can modify the value corresponding to a key by using the syntax:**

dictionary[key] =value

Eg: student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }

student ["mark"] = 290

Now,student = { "RollNo" : 10 , "Name" : "John" , "mark" : 290}

DOWNLOADED FROM KTUNOTES.IN

DICTIONARY MEMBERSHIP OPERATIONS

- We can test if a key is in a dictionary or not using the operators **in** and **not in**.
- If key is present , in operation returns true else returns false
- If key is not present not in operator returns true else returns false
- The membership test is only for the keys and not for the values.

```
squares = {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

if (1 in squares):
    print "Present"
else:
    print "Not Present"
```

If key 1 is present in the dictionary , display"Present"
Otherwise display "Not Present"

Output :Present

```
if (1 not in squares):
    print " Not Present"
else:
    print "Present"
```

If key 1 is not present in the dictionary , display"Not Present" Otherwise display "Present"

DOWNLOADED FROM KTUNOTES.IN

DELETING DICTIONARY ELEMENTS

- **del statement can be used if we know exactly which element is deleting.**

Syntax: del dictionaryname[key] will delete the element corresponding to that key

Eg: student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }

del student["mark"] will delete the item with key "mark"

student = { "RollNo" : 10 , "Name" : "John" }

dict1 = {1:100 ,2:200,3:300}

del dict1[2]

dict1 = {1:100 ,3:300]

DELETING AN ENTIRE DICTIONARY

Syntax: del dictionaryname

dict1 = {1:100 ,2:200,3:300}

del dict1 will delete the entire dictionary.

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

DICTIONARY METHODS(FUNCTIONS)

✓ **Used to perform various dictionary operations**

len()

❑ **This method is used to find the length of the dictionary(ie no.of items in the dictionary)**

Syntax: len(Dictionary name)

Eg: student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }

l=len(student) l=3

keys()

❑ **This method is used to return a list of keys from the Dictionary .**

Syntax: dictionary name. keys()

Eg: student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }

a= student. keys()

a= ["RollNo" , "Name" , "mark"]

DOWNLOADED FROM KTUNOTES.IN

values()

- This method is used to return a list of values from the Dictionary .

Syntax: dictionary name. values()

Eg: student = { “RollNo” : 10 , “Name” : “John” , “mark” : 280 }

a= student. values()

a=[10 , “John” , 280]

items()

- This method is used to return a list of (key,value) pairs from the Dictionary .

Syntax: dictionary name. items()

Eg: student = { “RollNo” : 10 , “Name” : “John” , “mark” : 280 }

a= student. items()

a= [(“RollNo” ,10) , (“Name”, “John”) , (“mark” ,280)]

DOWNLOADED FROM KTUNOTES.IN

pop()

- This method is used to remove a particular item in a Dictionary and returns it's value.

Syntax: dictionary name. pop(key)

Eg: 1.student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }

t= student. pop("mark") will delete the item with key "mark" t=280

Now, student = { "RollNo" : 10 , "Name" : "John" }

2.dict1 = {1:100 ,2:200,3:300}

t= dict1.pop(2) will delete the item with key 2 t=200

Now, dict1 = {1:100 ,3:300]

popitem()

- This method is used to remove and return an arbitrary item in a Dictionary .[Python2]

Syntax: dictionary name. popitem()

Eg: student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }

t= student. popitem()

Note: In Python3, It removes the last item in the dictionary

Note : We can use del statement also to delete an element from dictionary

DOWNLOADED FROM KTUNOTES.IN

clear()

- This method is used to remove all the items in a Dictionary at once.

Syntax: dictionary name. clear()

Eg: student = { “RollNo” : 10 , “Name” : “John” , “mark” : 280 }

student. clear() will remove all the items in the dictionary

sorted()

- This method is used to return a sorted list of keys from the Dictionary

Syntax: sorted(dictionary name) [Ascending order sorting]

a = sorted(dictionary name,reverse=True) [descending order sorting]

Eg: student = { 10:”John” , 1: “Ann” , 8: “James” }

a= sorted(student) b=sorted(student , reverse=True)

a= [1,8,10] b=[10,8,1]

DOWNLOADED FROM KTUNOTES.IN

Eg: student = { 8:"John" , 10: "Ann" , 1: "James" }

Display the details in ascending order of roll numbers

```
a= sorted(student)
for  k in a:
        print   k, student[k]
```

Output

```
1    James
8    John
10   Ann
```

Ktunotes.in

Eg: student = { 8:"John" , 10: "Ann" , 1: "James" }

Display the details in descending order of roll numbers

```
a= sorted(student,reverse=True)
for  k in a:
        print   k, student[k]
```

Output

```
10  Ann
8   John
1   James
```


DOWNLOADED FROM KTUNOTES.IN

update ()

- ✓ It is used to update the dictionary with the key - value pairs or with another dictionary.
- ✓ It inserts key-value pair if it is not present.
- ✓ It updates the value corresponding to the key if the key-value pair is already present in the dictionary.

Syntax :

1. dictionary name .update(key = value) – Update with key-value pair

Eg1: data ={“a”:10, “b”:20}

data.update(“c”=30) → data ={“a”:10, “b”:20,”c”:30}

Eg2: data ={“a”:10, “b”:20}

data.update(“b”=30) → data ={“a”:10, “b”:30}

2. dictionary name.update(dictionary2) –Update with dictionary2

Eg1: data ={“a”:10, “b”:20} data2={ 5 : 50 , 1:100}

data.update(data2) → data ={“a”:10, “b”:20,” 5:50 ,1:100}

DOWNLOADED FROM KTUNOTES.IN

DICTIONARY COPYING

1.Using Assignment operator(=)

```
d1={1:1,2:4 ,3:9}
d2=d1
d1[4] = 16
print “d1=“, d1 , “\nd2 =“ ,d2
```

Here d1 and d2 represent same memory locations where the dictionary values are stored. So when one variable is updated the change will be reflected in other variable also.

Output

```
d1={1:1,2:4 ,3:9 ,4:16}
d2={1:1,2:4 ,3:9 ,4:16}
```

Ktunotes.in

2.Using copy() function

```
d1={1:1, 2:4 , 3:9}
d2=d1.copy()
d1[4] = 16
print “d1=“, d1 , “\nd2 =“ ,d2
```

Here a copy of dictionary values are stored in d2(in different memory locations) So when one variable is updated it will not affect the other variable.

Output

```
d1={1:1,2:4 ,3:9 ,4:16}
d2={1:1, 2:4 , 3:9 }
```


DOWNLOADED FROM KTUNOTES.IN

2019 SCHEME UNIV Q- DICTIONARIES

b. Write a Python program to create a dictionary of roll numbers and names of five students. Display the contents of dictionary in alphabetical order of names. (6)

Suppose name is used as key(Assume that names are unique) . So we need to sort in asc order of keys

```
student={}
for i in range(1,6):
    r= input("enter roll no:")
    s=raw_input("enter name:")
    student[s]=r
print student
t=sorted(student)
for  i  in t:
   print  i, student[i]
```

for loop executes 5 times. so we can enter detais of 5 students .

student[s]= r means the key value pair(s,r) is inserted in the dictionary student. ie name is used as key and roll no as value corresponding to that key

The keys of dictionary(student) are sorted and stored in the list 't'

Traverse list 't' . here 'i'represents each element in the list (ie keys in the dictioary in sorted order)

we display i and student[i] ie key (name) and value (roll no)

Ktunotes.in

Illustrate when to use list, tuple and dictionary with example. (3)

List : ordered collection of items.It is mutable ie we can change a list (insert,delete,update elements)

Tuple: ordered collection of items. It is Immutable ie we can't change a tuple

Dictionary: unordered collection of items.Each item has a key and a corresponding value that is expressed as a pair (key: value.

When you want to use immutable ordered collection of elements use tuple.

When you want to use mutable ordered collection of elements use list

when you want to use mutable unordered collection of elements and you have a unique set of keys that map to values use dictionary

DOWNLOADED FROM KTUNOTES.IN

OR

14. (a) How to create Dictionary in Python? Write a Python program to read and display a sparse matrix using dictionary. (7)

(Sparse matrix is is a matrix for which majority of the elements are zero. We waste a lot of memory space when we express it with a normal 2-dimensional array.So we aim to store only the non-zero elements)

A Dictionary is created by enclosing the all the items inside curly brackets ({}) separated by commas

Syntax : Dictionary name={ key1 : value1 , key2 : value2 , key3 : value3 ,…….}

Eg: weekdays= {1: "Sunday", 2: "Monday ", 3: "Tuesday ", 4: "Wednesday"}

To read a sparse matrix using dictionary we read the non zero element and it's row and column indices

```
m=input("Enter no.of rows of sparse mat: ")
n=input("Enter no.of columns: ")
matrix1=[]
print "\nEntering the Elements for sparse matrix1....."
for i in range(0,m):
        a=[]
        for j in range(0,n):
                k=input()
                a.append(k)
    matrix1.append(a)

D={}
for i in range(0,m):
        for j in range(0,n):
                if(matrix1[i][j] !=0):
                    D[(i,j)] = matrix1[i][j]
Print "Sparse Matrix in Dictionary representation:" ,D
```

The non-zero elements are added to the dictionary .
Key : tuple representing row index and column index of nonzero value.
value : The nonzero value

For eg: if row and col index of nonzero value 7 are 1 and 3 resp: then the dictionary entry corresponding to that non zer value is (1,3) : 7

ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

21 a) What is a dictionary? Explain any five dictionary operations with examples. (7)

- **A Dictionary is an unordered collection of items.**
- **Each item has a key and a corresponding value that is expressed as a pair (key: value).**
- **Keys are unique within a Dictionary while values may not be unique.**

- **The values can be of any type but the keys must be of an immutable datatype such as numbers, strings or tuples.**

Eg: **student = { "RollNo" : 10 , "Name" : "John" , "mark" : 280 }**

dictionar operations (Briefly explain any 5)

1. **Dictionary Creation**
2. **Displaying dictionary**
3. **Dictionary Traversal**
4. **Updating dictionary elements**
5. **Deletion of Dictionary elements**
6. **Membership operations**

2015 SCHEME REPEATED UNIV Q

12. Describe the Dictionary methods with example (3)

Ktunotes.in

Refer Note.

keys(),values().items(),clear(),pop() etc

(b) Write a python program to create a dictionary of phone numbers and names of five persons. Display the contents of the dictionary in alphabetical order of names. (7)

Suppose name is used as key (Assume that names are unique) . So we need to sort in asc order of keys

```
D={}
for i in range(1,6):
    s=raw_input("enter name:")
    p= input("enter phone no:")
    D[s]=p
print  D
t=sorted(D)
for  k  in t:
    print  k, D [k]
```

DOWNLOADED FROM KTUNOTES.IN

13. Assume that the variable *data* refers to the dictionary {"b":34,"a":67}. Write the expressions that perform the following tasks

a) Replace the value at key "b" with negation of the value.

b) Add key:value pair "c":56 to *data*.

c) Remove the value at key "a" in *data*. (3)

a) data["b"] = -data["b"]

b) data["c"] = 56

c) del data["a"] or data.pop("a")

The elemets of dictionary are accessed by using keys.

dictionaryname[key] will give the value corresponding to that key

Q Explain the data structure, dictionary in Python using an example. How does the Dictionary operations- "del" , "len" , "keys ", "items" and "has_key" operations work. Explain with examples. (7)

Refer Note.

has_key : Used to check whether a key is present in the dictionary.This method returns true if specified key is present in the dictionary, else returns false.

Syntax: dictionary.has_key(key)

Eg: Data = { 'A': 'Apple', 'B': 'Banana', 'C': 'Cherry' }

print (Data.has_key('B')) output : True

Q 13. Let farm={'Sheep':5,'Cows':2,'Goats':10} be a dictionary. Write the statements for following operations. (3)

i) To add the key value pair ('Ducks':8)

ii) To display the number of items in the dictionary.

iii) To remove the key value pair ('Cows':2)

i) farm["Ducks"] = 8

ii) print len(farm)

iii) del farm["Cows"] or farm.pop("Cows")

DOWNLOADED FROM KTUNOTES.IN

13. Let student = {'John':50,'Tom':60, 'Nina':82} be a dictionary. Discuss the output obtained after executing the following statements:

```
newstudent = student
newstudent['Tom'] = 45
print newstudent
print student
```

(3)

Here newstudent and student represent same memory locations where the dictionary values are stored. So when one variable is updated the change will be reflected in other variable also.

{'John' :50 , 'Tom' : 60 , 'Nina' : 82 , 'Tom' : 45 }

{'John' :50 , 'Tom' : 60 , 'Nina' : 82 , 'Tom' : 45 }

Q

b) Create a dictionary named 'stock'. Add the following elements to stock and perform the following operations.

pencil - 400, pen - 1000, eraser- 200,
ink -50

i. Print stock.
ii. Delete ink and print stock.
iii. Explain keys and key- value pairs find the number of key-value pairs and identify the keys. (7)

Ktunotes.in

```
stock ={}
stock['pencil']=400
stock['pen']=1000
stock['eraser']=200
stock['ink']=50
```

i) print stock

ii) del stock['ink']
print stock

iii) A Dictionary is an unordered collection of items.Each item has a key and a corresponding value that is expressed as a pair (key: value).
The (key,value) pairs can be written in any order.
Keys are unique within a Dictionary while values may not be unique.
The values can be of any type but the keys must be of an immutable datatype such as numbers, strings or tuples.

- To find no. of key-value pairs(no.of elements in the dictionary) , we can use len(stock)
- Keys : pencil,pen,eraser,ink
- To display a list of keys , use the statement print stock.keys()

DOWNLOADED FROM KTUNOTES.IN

23. a) A book shop details contains the Title of book and Number of copies of each title. As books are added to shop the number of copies in each should increase and as books are sold the number of copies in each should decrease. Implement this scenario using Dictionary data type in Python (7)

```
books = {"c":100,"c++":200,"python":150}
while (True):
    print  "Menu\n 1.Add books\n 2.Sell books\n 3.Exit"
    a=input("Enter your choice:")
    if a==1:
        t=raw_input("Enter title:")
        c=input("Enter no.of copies:")
        if t in books:
            books[t]=books[t]+c
        else:
            books[t]=c
        print  books
    elif a==2:
        t=raw_input("Enter title:")
        c=input("Enter no.of copies:")
        if t not in books:
            print  "\n not available!!!! \n"
        elif c>books[t]:
            books[t]=0
        else:
            books[t]=books[t]-c
        print  books
    elif a==3:
        break
    else:
        print    "\n\tEnter valid choice(1/2/3)!!!!"
```

stock repr as dictioary. book title is used as key and no of copies that book is used as value

Display choice menu

For adding a new book in the stock, enter title and no of copies of that book.

If the book is already present in the stock, no. of copies are incremented by c.

Since title is used as key and no of copies as value, books[t] means no of copies of the book with title t

If the book is not already present in the stock, the title and it's no of copies are added as a new key,value pair

For selling ,first we enter title and no of copies of that book. requested by a customer

Then check whether the book requested by customer is already present in the stock.

If present next we check whether the requested no of copies are available in the stock.

If req no of copies (c) > no of copies already in the stock (books[t]) then no of copies in the stock becomes 0. Because all the copies will be given to customer

If req no of copies (c) <= no of copies already in the stock (books[t]) then no of copies in the stock is decremented by 'c' because 'c' copies will be given to customer

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

2015 SCHEME OTHER UNIV Q

Q

23 a) Create a dictionary of acc.no and balance of n account holders from console. As amount is deposited, the balance should increase and as amount is withdrawn the balance should decrease. Write a program using dictionary datatype in python for the above. (8)

same pattern qn as book shop details

```
Data={}
n=input("Enter no.of account holders:")
for i in range(1,n+1):
    a=input("enter account no:")
    b= input("enter account balance:")
    Data[a]=b

while (True):
        print   "Menu\n 1.Deposit\n 2.Withdraw\n 3.Exit"
        c=input("Enter your choice:")
        if c==1:
                ac=input("Enter account no:")
                amt=input("Enter amount to deposit:")
                Data[ac] =Data[ac]+amt
                print   Data
        elif c==2:
                ac=input("Enter account no:")
                amt=input("Enter amount to withdraw:")
                if  amt>books[ac]:
                        print "Insufficient Balance!!"
                else:
                        Data[ac] =Data[ac]-amt
                print   Data
        elif c==3:
                break
        else:
                print    "\n\tEnter valid choice(1/2/3)!!!!"
```

DOWNLOADED FROM KTUNOTES.IN

Q 21 a) Create a dictionary with *items* and *number of units* of each item available in the stock. Write a Python program to show the stock updation as and when the units of the existing items are added or deleted. (7)

same pattern qn as book shop details

```
items = {"Laptop":100,"MobilePhone":200,"Camera":150}
while (True):
        print   "Menu\n 1.Add Items\n 2.Delete Items\n 3.Exit"
        a=input("Enter your choice:")
        if a==1:
                i=raw_input("Enter item:")
                n=input("Enter no.of  units:")
                items[i]=items[i]+n
                print   items
        elif a==2:
                i=raw_input("Enter item:")
                n=input("Enter no.of  units:")
                if   n>items[i]:
                        items[i]=0
                else:
                        items[i]= items[i]-n
                print  items
        elif a==3:
                break
        else:
                print    "\n\tEnter valid choice(1/2/3)!!!!"
```

ktunotes.in

Q Explain any three methods of Dictionary type and write a Python program to display the frequency of each word in a string. Use Dictionary to store the word-frequency pairs. (7)

Dictionary methods : refer Note

DOWNLOADED FROM KTUNOTES.IN

```
s=raw_input("Enter the string")
x=s.split(" ")
print x
d={}
for w in x:
   if w not in d:
      d[w]=1
   else:
      d[w]=d[w]+1
print d
```

Eg: s="hello are u there hello how are u"

x=['hello','are','u','there','hello','how','are','u']

d={}

w=hello d['hello']=1

w=are d['are']=1

w= u d['u] =1

w=there d['there']=1

w=hello d['hello']=1+1=2

w=how d['how']=1

w='are' d['are']=1+1=2

w='u' d['u']=1+1=2

d={'hello' :2, 'are':2, 'a' :2 ,'there':1,.....}

Q 13 How can you create a fresh copy of a dictionary? Explain with example. (2)

Ans) Using copy() function

```
d1={1:1, 2:4 , 3:9}
d2=d1.copy()
```

d2={1:1, 2:4 , 3:9}

When d1 is modified it will not affect d2

```
d1[4] = 16
print "d1=", d1 , "\nd2 =" ,d2
```

Then : d1={1:1, 2:4 , 3:9 , 4:16} d2={1:1, 2:4 , 3:9}

Q10 For the dictionary given below, identify the key-value pairs. Also give the Python statements to print keys and values of the dictionary separately. (3)

Dict=['name':'john','age':25,'salary': 28000]

Key	value
name	John
age	25
salary	28000

To print keys

```
print Dict.keys()
```

To print values

```
print Dict.values()
```

DOWNLOADED FROM KTUNOTES.IN